

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



CSC10014 - TƯ DUY TÍNH TOÁN
HKII: 2025 - 2026
HỆ THỐNG LƯU TRÚ
DU LỊCH THÔNG MINH

Lớp : CQ2024/6
Nhóm thực hiện : Nhóm 12
Giáo viên hướng dẫn : Hồ Tuấn Thanh
GVHDTH : Mai Anh Tuấn
Trợ giảng : Phạm Nguyễn Sơn Tùng

Thành Viên:

24120033: Đào Tiến Đạt.
24120040: Dương Hoài Đức.
24120304: Trần Thị Bích Hạnh.
24120310: Nguyễn Đình Hiếu.
24120483: Hoàng Văn Trường.

THÀNH PHỐ HỒ CHÍ MINH, NGÀY 20/6/2026

Lời Cảm Ơn

Trong quá trình thực hiện đồ án môn học, nhóm chúng em đã nhận được rất nhiều sự hỗ trợ và hướng dẫn quý báu từ các thầy cô và bạn bè.

Trước hết, chúng em xin gửi lời cảm ơn chân thành đến thầy **Hồ Tuấn Thanh** – giảng viên phụ trách học phần, đã cung cấp những kiến thức nền tảng cũng như tạo điều kiện để chúng em có cơ hội tiếp cận với các công nghệ và quy trình phát triển ứng dụng thực tế. Chúng em cũng xin chân thành cảm ơn thầy **Mai Anh Tuấn** – giảng viên hướng dẫn thực hành cùng các thầy cô trợ giảng đã tận tình hỗ trợ, giải đáp các thắc mắc trong quá trình học tập và thực hiện đồ án.

Thông qua học phần này, các thành viên trong nhóm không chỉ tích lũy được những kiến thức chuyên môn quý báu mà còn rèn luyện được tư duy giải quyết vấn đề, kỹ năng làm việc nhóm và khả năng vận dụng kiến thức vào các bài toán thực tế. Những kinh nghiệm thu được trong quá trình thực hiện đề tài sẽ là nền tảng quan trọng cho việc học tập và nghiên cứu trong tương lai.

Mặc dù đã nỗ lực hoàn thành đồ án với tinh thần học hỏi và trách nhiệm cao nhất, song do kiến thức và kinh nghiệm thực tiễn còn hạn chế nên bài làm khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được những ý kiến đóng góp và nhận xét quý báu từ Thầy/Cô để có thể tiếp tục hoàn thiện kiến thức và kỹ năng của mình.

Cuối cùng, chúng em xin kính chúc quý Thầy/Cô luôn dồi dào sức khỏe, hạnh phúc và gặt hái được nhiều thành công trong sự nghiệp giảng dạy và nghiên cứu.

Trân trọng,
Các thành viên Nhóm 12

Table of Contents

1	Group Member	4
2	Idea	5
2.1	Problem Statement	5
2.2	Application Vision	5
2.3	Target Users	5
3	Problem Analysis	6
3.1	User Journey	6
3.2	Computational Problem Decomposition	6
3.3	Constraints, Assumptions, and Criteria	7
4	System Overview	8
4.1	Overall Architecture	8
4.2	Core Features	9
4.3	Feature Illustrations	9
4.4	Breakthrough Points	11
5	Pattern Recognition	12
5.1	Purpose and Approach	12
5.2	Application of Pattern Recognition Thinking	12
5.3	Summary	15
6	Abstraction	17
6.1	Role of Abstraction in System Architecture	17
6.2	Abstracted Components	17
6.3	Architectural Benefits and Balance Level	18
7	System / Algorithm Design	19
7.1	Architectural Design Goals	19
7.2	Overall Architecture	19
7.3	C4 Level 1: System Context	20
7.4	C4 Level 2: Container Architecture	20
7.5	Backend Modular Architecture	21
7.6	Frontend Architecture	21

7.7	Data Architecture	22
7.8	Key Runtime Workflows	22
7.9	Security and Access Control	25
7.10	Runtime Configuration and Environment Strategy	25
7.11	Architectural Decisions and Trade-offs	26
7.12	Current Limitations and Future Improvements	26
7.13	Summary	26
8	Implementation	27
8.1	Implementation Strategy	27
8.2	From Design to Implementation	27
8.3	Team Implementation Workflow	27
8.4	Core Implemented Parts	28
8.5	Technical Challenges and Solutions	28
8.6	Quản lý ngữ cảnh giao tiếp cho mô hình ngôn ngữ	29
8.7	Summary	30
9	Testing	30
9.1	Testing Strategy and Methodology	30
9.2	Execution Results and Statistics	31
9.3	Quality Assurance and Deep Dive Evaluation	31
9.3.1	Điểm mạnh	31
9.3.2	Điểm hạn chế	32
10	Logbook: Plan and Actual Workload	32
10.1	Detail of Plan Workload	33
10.2	Actual Workload	33
10.3	Workload Distribution	34
10.4	List of tools used	34
11	AI usage and declaration	34
12	Conclusion and Future Work	34
12.1	Conclusion	35
12.2	Team Reflections and Lessons Learned	35
12.3	Future Development Directions	35

1. Group Member

STT	MSSV	Họ và Tên	Vai trò / Nhiệm vụ chính
1	24120033	Đào Tiến Đạt	
2	24120040	Dương Hoài Đức	
3	24120304	Trần Thị Bích Hạnh	
4	24120310	Nguyễn Đình Hiếu	
5	24120483	Hoàng Văn Trường	

2. Idea

2.1. Problem Statement

Hiện nay, việc tìm kiếm một nơi lưu trú phù hợp khi đi du lịch không hề đơn giản. Người dùng thường phải cân nhắc nhiều yếu tố như giá cả, vị trí, tiện nghi, chất lượng dịch vụ và các đánh giá từ những khách hàng trước đó. Quá trình này vẫn còn gặp nhiều khó khăn với các **điểm đau** cốt lõi:

- ❌ **Thông tin phân mảnh và Rời rạc:** Dữ liệu về khách sạn, homestay rải rác trên nhiều nền tảng, khiến người dùng mất nhiều thời gian tổng hợp và so sánh.
- ❌ **Bộ lọc cứng nhắc và Thiếu cá nhân hoá:** Các công cụ truyền thống chỉ cho phép lọc theo các tùy chọn có sẵn, không hỗ trợ tìm kiếm linh hoạt dựa trên nhu cầu tự nhiên và đặc thù của người dùng.
- ❌ **Độ tin cậy của dữ liệu thấp:** Đánh giá từ người dùng trước thường mang tính chủ quan, có thể bị thao túng bằng các đánh giá giả mạo và thiếu tính cập nhật thực tế tại thời điểm hiện tại.
- ❌ **Quá tải thông tin và Tốn thời gian:** Người dùng mất quá nhiều thời gian để đọc và đối chiếu thông tin từ nhiều nguồn trước khi có thể đưa ra quyết định chính xác.

💡 Giải pháp đề xuất

Nhóm xây dựng hệ thống tìm kiếm và gợi ý lưu trú thông minh, cho phép người dùng mô tả nhu cầu bằng **ngôn ngữ tự nhiên**. Hệ thống sẽ phân tích yêu cầu, tìm kiếm các địa điểm phù hợp, đánh giá, xếp hạng và đưa ra những gợi ý trực quan, giúp người dùng ra quyết định nhanh chóng.

2.2. Application Vision

Dự án không chỉ nhằm xây dựng một nền tảng thương mại thông thường, mà định hình lại cách người dùng tương tác với dữ liệu. Thông qua việc ứng dụng AI và kỹ thuật **RAG**, hệ thống kỳ vọng tạo ra một tiền đề cho tương lai, nơi AI đóng vai trò như một người điều phối đáng tin cậy.

2.3. Target Users

- 👤 **Khách du lịch tự túc:** Cần tìm kiếm nhanh chóng, so sánh giá để tối ưu hóa trải nghiệm.
- 👤 **Người dùng On-site:** Đang lưu trú tại địa điểm, sẵn sàng đóng góp thông tin thực tế và trả lời câu hỏi cộng đồng.
- 👤 **Người ưu tiên cá nhân hóa:** Thích tương tác với AI thay vì tốn hàng giờ với các bộ lọc phức tạp.

3. Problem Analysis

3.1. User Journey

Thay vì tiếp cận theo hướng kỹ thuật khô khan, nhóm thiết kế hệ thống thông qua chuỗi câu hỏi dẫn dắt từ góc nhìn người dùng:

- ❓ Người dùng sẽ làm gì với ứng dụng của tôi?**
→ Người dùng cần tìm địa điểm lưu trú.
- ❓ Người dùng có thể tìm kiếm địa điểm lưu trú như thế nào?**
→ Thiết kế giao diện tìm kiếm hoặc phương thức tìm kiếm linh hoạt.
- ❓ Vậy làm sao để phương thức tìm kiếm có thể cho ra kết quả tốt?**
→ Chúng ta sẽ đánh giá trên các tiêu chí, như vậy chúng ta cần trích xuất ý định của người dùng.
- ❓ Vậy việc trích xuất nên được thực hiện thế nào?**
→ Có thể cho phép người dùng chọn hoặc thông qua tác tử AI được cài đặt.
- ❓ Nếu người dùng sau khi đã có các địa điểm phù hợp với nhu cầu của họ, làm cách nào để hệ thống có thể giúp người dùng ra quyết định cuối cùng như thế nào?**
→ Có thể cài đặt tính năng giúp người dùng so sánh các địa điểm, hỗ trợ ra quyết định cuối.
- ❓ Vậy nếu người dùng muốn tìm hiểu sâu hơn về một địa điểm thì sao?**
→ Cài đặt tính năng AI đưa ra phân tích chi tiết về một địa điểm cụ thể, cho phép người dùng lựa chọn các tiêu chí để phân tích.
- ❓ Việc sử dụng LLM có thể không đạt được độ chính xác cao, liệu nguồn thông tin nào uy tín hơn không?**
→ Sử dụng nguồn thông tin từ chính người dùng, có thể áp dụng chính sách 'người dùng onsite' để tăng độ tin cậy.
- ❓ Vậy với các tính năng như trên, có thể tối giản giao diện để người dùng có thể tương tác nhanh chóng và dễ dàng, phù hợp với nhiều người dùng ở các độ tuổi, trình độ khác nhau hay không?**
→ Sử dụng LLM đóng vai trò điều phối trung gian, người dùng sẽ tương tác trực tiếp với LLM, LLM sẽ trích xuất ý định người dùng và tiến hành các yêu cầu một cách tự động.

3.2. Computational Problem Decomposition

Từ nhu cầu thực tế, hệ thống được phân rã thành các module kỹ thuật chuyên biệt:

- ⚙️ Intent Parsing:** LLM chuyển đổi ngôn ngữ tự nhiên thành JSON cấu trúc.

- ⚙️ **Data Aggregation:** Truy xuất đồng thời DB nội bộ và API đối tác.
- ⚙️ **Semantic Search:** Số hóa văn bản thành vector nhúng (với pgvector).
- ⚙️ **Workflow Engine:** Xây dựng khung làm việc cứng kiểm soát AI.

3.3. Constraints, Assumptions, and Criteria

Để đảm bảo tính khả thi trong phạm vi đề án, hệ thống được thiết kế và đánh giá dựa trên các khía cạnh rõ ràng sau:

🔗 1. Ràng buộc

- ▶ **Về mặt kiến trúc:** Lựa chọn mô hình *Modular Monolith* để giảm thiểu chi phí quản lý và vận hành trong giai đoạn MVP, đồng thời vẫn giữ được tính phân tách cao giúp dễ dàng mở rộng sang Microservices sau này.
- ▶ **Về mặt tài nguyên:** Hệ thống chuyển giao toàn bộ tải tính toán AI nặng nề sang các API của bên thứ ba (như Groq). Nhờ đó, hệ thống lõi có thể chạy cục bộ mượt mà trên phần cứng cơ bản không yêu cầu GPU.
- ▶ **Về phạm vi hỗ trợ:** Ứng dụng ở giai đoạn này tập trung vào thị trường Việt Nam. Hệ thống tập trung xác định các địa điểm lưu trú trong nước, yêu cầu AI phải hỗ trợ tiếng Việt tốt và hiểu rõ ngữ cảnh văn hóa địa phương.

💡 2. Giả định

- ▶ Khả năng lập luận, phân tích ngữ nghĩa và đưa ra quyết định của Mô hình ngôn ngữ lớn là yếu tố khách quan, tính chính xác phụ thuộc vào API cung cấp chứ không bị giới hạn bởi hệ thống nội bộ.
- ▶ Dữ liệu thu thập từ các nhà cung cấp bên thứ ba (Goong, SerpAPI) luôn đảm bảo độ tin cậy về mặt nội dung và ổn định về thời gian hoạt động liên tục.

☰ 3. Tiêu chí đánh giá

- ▶ **Sự hài lòng của người dùng:** Đánh giá mức độ thoải mái, mức độ cá nhân hóa và sự hài lòng tổng thể khi người dùng trải nghiệm các tính năng tìm kiếm, trò chuyện cùng AI trên nền tảng.
- ▶ **Tính hữu dụng:** Ứng dụng phải giải quyết được triệt để bài toán tìm kiếm chỗ ở. Giao diện cần trực quan, dễ thao tác và có luồng tương tác mượt mà phù hợp với nhiều nhóm đối tượng.
- ▶ **Độ chính xác của thông tin:** Đảm bảo chất lượng dữ liệu trả về. Danh sách địa điểm phải sát thực tế, và đặc biệt câu trả lời phân tích chi tiết từ AI phải chuẩn xác, tuyệt đối không được xảy ra hiện tượng Hallucination.

4. System Overview

Hệ thống là nền tảng tìm kiếm và quản lý lưu trữ thông minh, kết hợp tra cứu truyền thống và sức mạnh Trí tuệ Nhân tạo. Dự án mang đến trải nghiệm đột phá qua giao diện hội thoại và tìm kiếm ngữ nghĩa.

4.1. Overall Architecture

Hệ thống được thiết kế theo mô hình **Modular Monolith** kết hợp với cơ chế điều phối tác tử AI.

❓ Tại sao chọn Modular Monolith?

Lý do nhóm ưu tiên sử dụng **Modular Monolith** là nhằm giảm thiểu tối đa sự phức tạp trong quá trình triển khai và vận hành hệ thống ở giai đoạn nguyên mẫu. Với kiến trúc này, toàn bộ hệ thống được đóng gói chung để dễ quản lý, nhưng bên trong lại được phân tách nghiêm ngặt thành các phân hệ nghiệp vụ độc lập. Cách thiết kế này giúp mã nguồn luôn rõ ràng, ngăn chặn sự chồng chéo logic, tiết kiệm tài nguyên hệ thống, và đặc biệt là tạo tiền đề thuận lợi để dễ dàng bóc tách thành các dịch vụ độc lập dưới dạng Microservices trong tương lai khi dự án thực sự cần mở rộng quy mô.

Về tổng thể, kiến trúc bao gồm ba tầng chính:

- 🏠 **Tầng Giao diện:** Tập trung tối ưu hóa trải nghiệm người dùng, cung cấp bản đồ tương tác, không gian hỏi đáp cộng đồng và đặc biệt là khung trò chuyện AI với khả năng nhận luồng phản hồi liên tục theo thời gian thực.
- 🏢 **Tầng Xử lý (Nghiệp vụ và AI):** Đóng vai trò là trung tâm điều phối, bao gồm các phân hệ quản lý người dùng, địa điểm và đánh giá. Điểm nhấn là phân hệ AI hoạt động như một cỗ máy điều phối quy trình công việc, chịu trách nhiệm phân tích ý định từ ngôn ngữ tự nhiên và tự động kích hoạt các công cụ tương ứng.
- 🗄️ **Tầng Dữ liệu:** Chịu trách nhiệm lưu trữ an toàn toàn bộ dữ liệu quan hệ của nền tảng. Đồng thời, tầng này cũng tích hợp khả năng lưu trữ và truy vấn các vector nhúng, làm cốt lõi cho tính năng tìm kiếm ngữ nghĩa và truy xuất dữ liệu tăng cường.

4.2. Core Features

STT	Tính Năng	Mô Tả Ngắn
1	 Tìm kiếm và Gợi ý	Trải nghiệm tìm kiếm đa chiều kết hợp linh hoạt giữa bộ lọc và hệ thống gợi ý cá nhân hóa.
2	 Trợ lý Ảo AI	Chatbot thông minh hỗ trợ tìm kiếm và chất lọc thông tin lưu trữ qua hội thoại tự nhiên.
3	 Quản lý Cộng đồng	Không gian tương tác, hỏi đáp và chia sẻ nhận xét trực tiếp tại trang của từng địa điểm.
4	 Xác nhận On-site	Tính năng check-in độc đáo để xác nhận người dùng đang có mặt tại địa điểm thực tế.
5	 Đóng góp Nội dung	Trao quyền cho người dùng tải lên hình ảnh và cập nhật tình trạng mới nhất của chỗ ở.
6	 Lưu trữ Địa điểm	Công cụ cá nhân để đánh dấu, lưu lại và so sánh các danh sách địa điểm yêu thích.

4.3. Feature Illustrations

Thay vì tiếp cận theo hướng kỹ thuật, hệ thống được thiết kế dựa trên quá trình tối ưu hóa trải nghiệm thực tế của người dùng thông qua các tính năng sau:

 1. Trải nghiệm Tìm kiếm Linh hoạt

Hệ thống khắc phục hạn chế của các bộ lọc tìm kiếm truyền thống bằng cách cho phép người dùng nhập trực tiếp các yêu cầu phức tạp dưới dạng ngôn ngữ tự nhiên. Ví dụ: *"Tìm một home-stay yên tĩnh ở Đà Lạt cho gia đình 4 người, có bếp nướng BBQ và giá dưới 2 triệu."* Thông qua khả năng phân tích ngữ nghĩa, hệ thống sẽ trích xuất chính xác các tiêu chí và trả về danh sách địa điểm phù hợp nhất.



 2. Hỏi đáp Trực tiếp cùng Trợ lý Ảo

Illustration (0.6 width)
(Chat AI)

Nhằm giảm thiểu thời gian đọc và tổng hợp thông tin từ hàng loạt đánh giá dài, người dùng có thể đặt câu hỏi trực tiếp cho AI tại trang thông tin của điểm lưu trú. Ví dụ: "*Khách sạn này ăn sáng có đa dạng không? Tốc độ mạng wifi có ổn định không?*". Trợ lý ảo sẽ tự động phân tích các đánh giá thực tế từ cơ sở dữ liệu để đưa ra câu trả lời khách quan, ngắn gọn và bám sát thắc mắc của người dùng.

✔ 3. Chống Thông tin Sai lệch qua cơ chế “On-site”

Để giải quyết rủi ro sai lệch thông tin giữa quảng cáo và thực tế, hệ thống tích hợp tính năng xác thực "Check-in" dành riêng cho những khách hàng đang có mặt tại điểm lưu trú. Những hình ảnh, đánh giá hoặc câu trả lời được cung cấp bởi nhóm người dùng "On-site" này sẽ được gắn nhãn xác thực và ưu tiên hiển thị, qua đó nâng cao độ tin cậy của thông tin trên nền tảng.

Illustration (0.6 width)
(On-site)

👥 4. Tương tác Cộng đồng Trực tiếp

Illustration (0.6 width)
(Community)

Mỗi địa điểm trên nền tảng được tổ chức như một diễn đàn độc lập. Người dùng có thể đặt các câu hỏi đặc thù và nhận phản hồi trực tiếp từ những du khách đã hoặc đang lưu trú tại đó. Tính năng này tạo ra một vòng lặp chia sẻ thông tin liên tục, giúp dữ liệu luôn được cập nhật và phản ánh đúng tình trạng hiện tại của điểm đến.

4.4. Breakthrough Points

Các điểm đổi mới nổi bật

- ✓ **AI-Driven Workflow:** AI không chỉ sinh chữ. Nó là "Router" gọi API, tính toán logic và vẽ thẳng kết quả lên UI.
- ✓ **Semantic Search và Vector DB:** Khám phá chỗ ở thông qua độ tương đồng Cosine của ngữ nghĩa, vượt ra khỏi giới hạn so khớp từ khóa cứng nhắc.
- ✓ **On-site Insights:** Giải quyết triệt để sự sai lệch giữa quảng cáo và thực tế bằng cách ưu tiên tương tác từ người dùng thực chứng.

5. Pattern Recognition

5.1. Purpose and Approach

Trong dự án này, Nhận diện mẫu đóng vai trò thiết yếu để định hình kiến trúc hệ thống. Thay vì giải quyết từng vấn đề một cách đơn lẻ, nhóm đã quan sát các trường hợp lặp lại trong quá trình vận hành, so sánh chúng, gom nhóm những điểm tương đồng và khái quát hóa thành các khuôn mẫu thiết kế hữu ích. Các phương pháp tổng quát như đối chiếu nhiều trường hợp, phân tích sự lặp lại và kiểm chứng bằng các kịch bản kiểm thử đã được áp dụng triệt để.

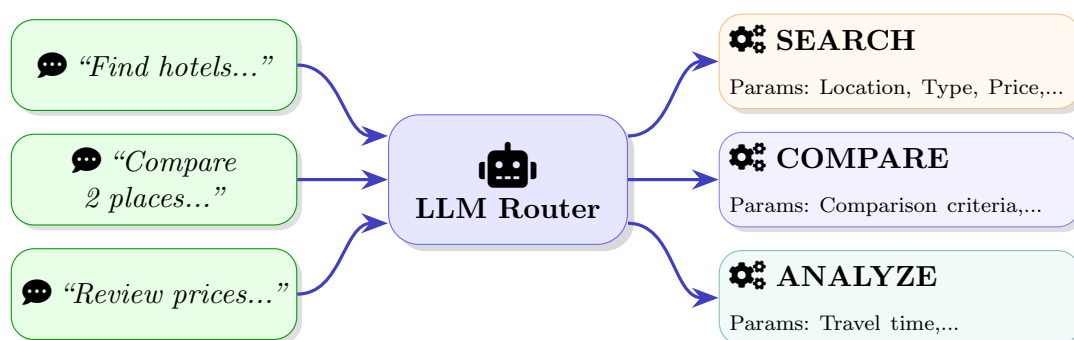
Việc lựa chọn cách tiếp cận này xuất phát từ đặc thù của hệ thống: phải tiếp nhận vô số câu lệnh tự nhiên đa dạng, vận hành nhiều luồng xử lý song song, tổng hợp thông tin từ nhiều nguồn và đối mặt với hàng loạt tình huống lỗi ngoại vi. Nếu xử lý từng trường hợp riêng lẻ, hệ thống sẽ trở nên rời rạc, thiếu đồng bộ và rất khó mở rộng. Nhờ đúc kết được 6 quy luật cốt lõi dưới đây, nhóm đã có cơ sở vững chắc để đưa ra các quyết định thiết kế kiến trúc chuẩn xác.

5.2. Application of Pattern Recognition Thinking

Q 1. Nhận diện mục đích tìm kiếm

Thực tế cho thấy, dù người dùng có cách diễn đạt đa dạng khi tìm kiếm nơi lưu trú, các câu lệnh của họ luôn xoay quanh những thông tin cốt lõi lặp lại như vị trí, ngân sách, loại chỗ ở, tiện nghi và các yêu cầu mềm khác.

Quy tắc tổng quát được rút ra là: mọi yêu cầu phức tạp đều có thể được bóc tách thành các thông tin cụ thể kết hợp với mục đích cốt lõi. Quy tắc này giúp nhóm thiết kế một hệ thống trí tuệ nhân tạo có khả năng tự động đọc hiểu câu văn tự nhiên của người dùng, từ đó phân tích và chuyển đổi thành những điều kiện tìm kiếm chuẩn xác để hệ thống có thể xử lý dễ dàng.

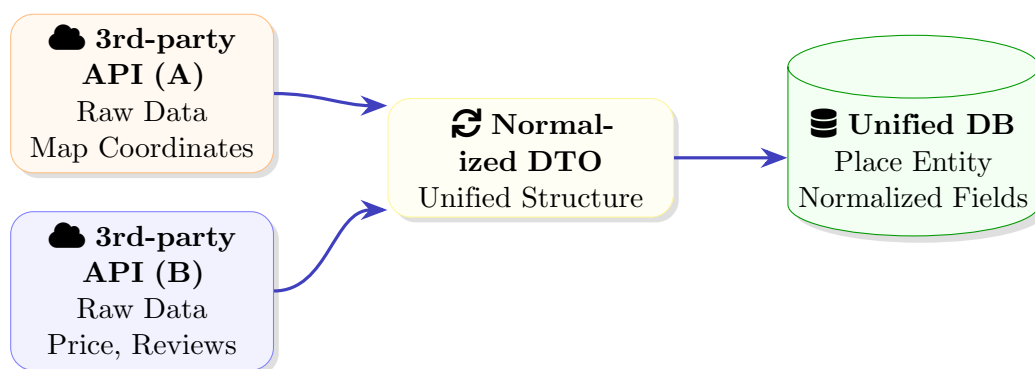


Hình 1: Query Pattern — Chuẩn hóa đa dạng câu hỏi tự nhiên thành các tham số định tuyến.

2. Nhận diện cấu trúc thông tin lưu trú

Trong quá trình thu thập thông tin từ nhiều nguồn khác nhau, nhóm phát hiện ra một sự tương đồng lớn về mặt nội dung. Dù định dạng dữ liệu ban đầu từ các bên cung cấp có sự khác biệt, mọi điểm lưu trữ đều chia sẻ những thông tin nền tảng giống nhau.

Dựa trên quy tắc này, nhóm quyết định thiết kế một cơ chế trung gian có nhiệm vụ thu gom, sắp xếp và đồng nhất mọi định dạng dữ liệu trước khi lưu trữ chính thức vào hệ thống cốt lõi.

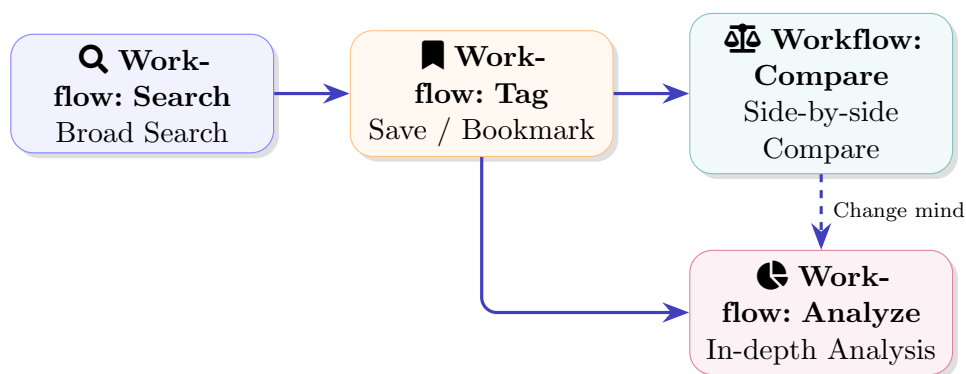


Hình 2: Data Pattern — Nhận diện mẫu thuộc tính từ các nguồn phân mảnh để ánh xạ về mô hình chuẩn hóa Place.

3. Nhận diện quy trình ra quyết định

Hành vi của người dùng trong quá trình lựa chọn chỗ ở hiếm khi diễn ra theo một đường thẳng. Thực tế cho thấy một thói quen lặp lại: Người dùng sẽ bắt đầu bằng việc tìm kiếm diện rộng, sau đó đánh dấu những nơi ưng ý, và cuối cùng là so sánh hoặc đánh giá chi tiết.

Từ đặc điểm này, nhóm đã xây dựng sẵn các quy trình xử lý cốt lõi được liên kết chặt chẽ với nhau nhằm hỗ trợ trọn vẹn toàn bộ quá trình cân nhắc và ra quyết định phức tạp của người dùng.



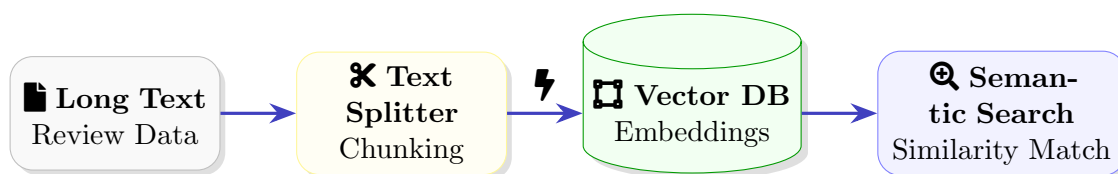
Hình 3: Decision-making Pattern — Khớp nối Workflow hệ thống với mẫu hành vi ra quyết định lặp lại.

★ 4. Nhận diện phương pháp xử lý lưu trữ thông tin

Trong thực tiễn, các luồng thông tin dạng ngôn ngữ tự nhiên như đánh giá, nhận xét hay mô tả của người dùng có mức độ phân hóa vô cùng đa dạng. Mặc dù không theo một cấu trúc cố định, những thông tin này lại đóng vai trò tối quan trọng: chúng thường xuyên được sử dụng làm ngữ cảnh hỗ trợ, cung cấp dữ liệu chi tiết cho Mô hình ngôn ngữ lớn nhằm tối ưu hóa và gia tăng độ chính xác cho câu trả lời.

Nhận thức được điều này, nhóm đánh giá việc thiết lập một khuôn mẫu chung để xử lý thống nhất các luồng ngôn ngữ tự do là một yêu cầu cấp thiết. Từ đó đã dẫn đến **ý tưởng then chốt**: tiến hành "số hóa" toàn bộ thông tin dạng ngôn ngữ tự nhiên.

Việc chuyển đổi câu chữ thành các dải số học không chỉ giúp máy tính hiểu được ý nghĩa sâu xa mà còn cho phép trích xuất chính xác các ngữ cảnh cần thiết nhất để phục vụ cho hệ thống xử lý.



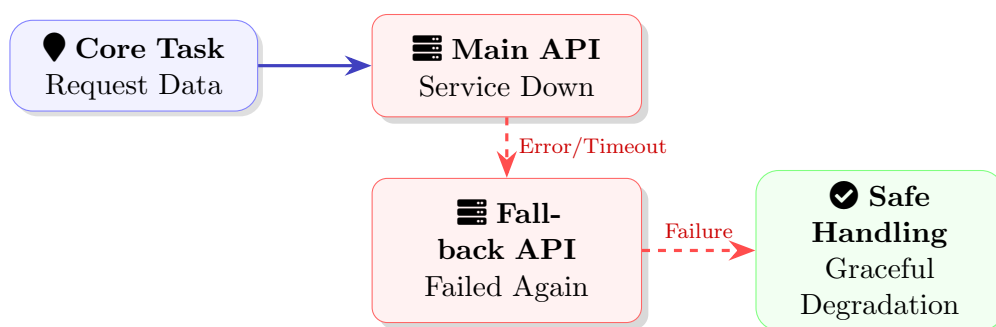
Hình 4: RAG Pattern — Xử lý văn bản tự do thành các khối thông tin truy xuất được bằng vector ngữ nghĩa.

⚠ 5. Nhận diện rủi ro hệ thống

Việc kết nối với các dịch vụ bên ngoài luôn tiềm ẩn những rủi ro ngất quảng như quá tải hoặc mất kết nối. Điều này dẫn đến **quy tắc thiết kế**: sự cố từ bên ngoài là không thể tránh khỏi, do đó hệ thống cốt lõi bắt buộc phải có khả năng tự bảo vệ và phục hồi.

Khi một dịch vụ chính gặp sự cố, hệ thống sẽ tự động kích hoạt cơ chế chuyển đổi sang dịch vụ dự phòng.

Trong trường hợp mọi phương án đều thất bại, hệ thống sẽ chủ động xử lý lỗi một cách khéo léo và trả về kết quả an toàn nhằm đảm bảo trải nghiệm của người dùng không bị gián đoạn.

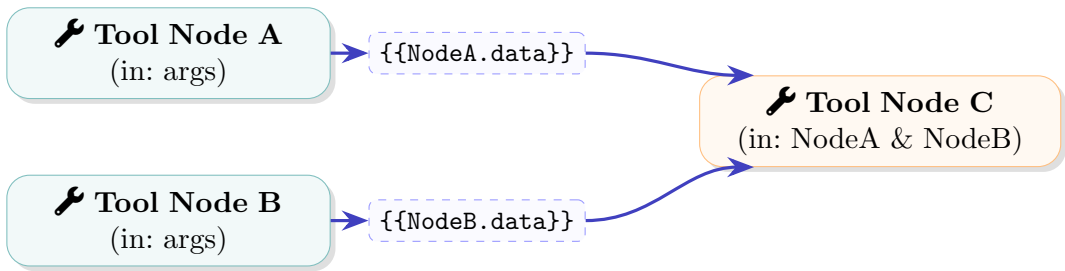


Hình 5: Failure Pattern — Cơ chế Fallback đối phó với mẫu lỗi lặp lại từ các nhà cung cấp API ngoại vi.

🔗 6. Nhận diện luồng liên kết workflow

Các quy trình xử lý phức tạp thực chất là một chuỗi các bước thực hiện, nơi kết quả của bước trước trở thành nguyên liệu đầu vào cho bước sau. Nhóm đã nhận diện được đặc điểm này và giải quyết bằng cách thiết lập một cơ chế truyền dữ liệu linh hoạt làm cầu nối giao tiếp giữa các tác vụ.

Quyết định thiết kế này giúp tạo ra một luồng công việc linh động, cho phép hệ thống dễ dàng lắp ráp, đảo vị trí hoặc thêm mới các thao tác hỗ trợ mà không làm phá vỡ cấu trúc tổng thể.



Hình 6: Workflow Tool Pattern — Nhận diện mẫu phụ thuộc dữ liệu bằng cách sử dụng String Template giữa các Tool.

5.3. Summary

Bảng 1: Pattern Recognition Summary

📋 Mẫu được nhận diện	💡 Quy tắc tổng quát	⚙️ Quyết định thiết kế
Query Pattern Cách diễn đạt đa dạng nhưng luôn xoay quanh các tiêu chí, ý định cốt lõi	Đa phần yêu cầu phức tạp đều có thể được phân tách thành mục đích chính và các điều kiện cụ thể.	Sử dụng LLM để đọc hiểu và chuyển đổi câu văn tự nhiên thành các tiêu chí tìm kiếm chuẩn xác.
Data Pattern Thông tin từ nhiều nguồn luôn có sự tương đồng về thuộc tính nền tảng.	Mọi thông tin thô dù khác biệt định dạng đều có thể được ánh xạ về một cấu trúc chung.	Xây dựng cơ chế trung gian để thu gom, làm sạch và đồng nhất toàn bộ dữ liệu trước khi lưu trữ.
Decision-making Pattern Quá trình chọn lọc thường đi qua nhiều bước cân nhắc, đối chiếu.	Hành vi ra quyết định hiếm khi đi đường thẳng mà là một chuỗi hành động lặp lại.	Cung cấp sẵn các luồng xử lý liên kết chặt chẽ (lưu trữ, so sánh, phân tích) để hỗ trợ quá trình cân nhắc.

Bảng 1: Pattern Recognition Summary (tiếp tục)

 Mẫu được nhận diện	 Quy tắc tổng quát	 Quyết định thiết kế
RAG Pattern Cơ chế xử lý đối với dữ liệu dạng ngôn ngữ tự nhiên	Các đoạn văn bản dài cần được phân tích theo ngữ nghĩa để dễ dàng trích xuất thông tin trọng tâm.	Số hóa văn bản thành các dải số học và sử dụng kỹ thuật tìm kiếm tương đồng để tự động giải đáp.
Failure Pattern Các dịch vụ bên ngoài thường xuyên gặp sự cố mất kết nối hoặc quá tải	Rủi ro từ ngoại vi là không thể tránh khỏi, hệ thống bắt buộc phải biết tự bảo vệ.	Thiết lập cơ chế tự động chuyển sang dịch vụ dự phòng hoặc xử lý lỗi khéo léo để không gây gián đoạn.
Workflow Tool Pattern Các quy trình phức tạp luôn vận hành theo chuỗi liên kết thông tin	Các tools được sử dụng có thể được nối với nhau để tạo ra một workflow hoàn chỉnh	Tạo ra cơ chế truyền dữ liệu linh hoạt, cho phép dễ dàng tháo lắp và sắp xếp các bước xử lý một cách trơn tru.

Tổng kết lại, các khuôn mẫu được nhận diện đã giúp nhóm biến những hiện tượng lặp lại trong thực tế thành các quy tắc thiết kế chuẩn mực. Hệ thống hoàn toàn không được thiết kế như một tập hợp các tính năng rời rạc, mà được tổ chức một cách có hệ thống xoay quanh chính các cấu trúc lặp lại của bài toán gốc. Việc áp dụng thành công tư duy nhận diện mẫu mang lại tác dụng to lớn: giúp toàn bộ kiến trúc trở nên nhất quán hơn, dễ dàng nâng cấp và mở rộng, tạo thuận lợi tối đa cho công tác kiểm thử, và quan trọng nhất là đáp ứng chính xác nhu cầu thực tiễn của người dùng.

6. Abstraction

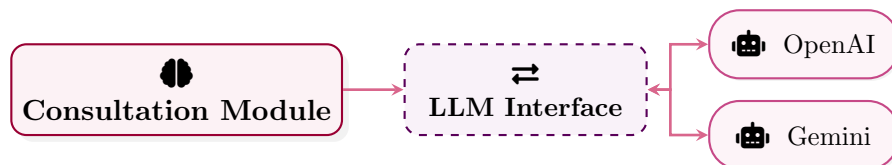
6.1. Role of Abstraction in System Architecture

Đây là một nền tảng lưu trữ du lịch thông minh, đòi hỏi sự phối hợp liên tục giữa nhiều thành phần có độ phức tạp cao như mô hình ngôn ngữ lớn LLM, dịch vụ bản đồ, giao tiếp dữ liệu địa điểm, cơ sở dữ liệu vector và các cơ chế xác thực. Nếu để hệ thống phụ thuộc trực tiếp vào đặc tả chi tiết của từng nhà cung cấp dịch vụ, toàn bộ kiến trúc sẽ trở nên cứng nhắc, khó mở rộng và dễ đứt gãy mỗi khi có sự thay đổi từ bên ngoài. Để giải quyết rào cản này, nhóm phát triển đã áp dụng mạnh mẽ tư duy trừu tượng hóa nhằm thiết lập các lớp khái niệm ổn định, che giấu hoàn toàn các chi tiết triển khai cụ thể phía sau. Tư duy này cho phép hệ thống giao tiếp với các dịch vụ cốt lõi dựa trên chức năng thiết yếu của chúng thay vì bị trói buộc vào một công nghệ hay nhà cung cấp nhất định.

6.2. Abstracted Components

Trong quá trình xây dựng kiến trúc hệ thống, tư duy trừu tượng hóa được áp dụng xuyên suốt trên nhiều khía cạnh khác nhau:

1. Năng lực xử lý ngôn ngữ: Hệ thống không bị ràng buộc vào một nhà cung cấp AI cụ thể nào. Ở mức kiến trúc, LLM đơn thuần được xem như một tác nhân cung cấp khả năng thấu hiểu ngôn ngữ tự nhiên, phân tích ý định người dùng và tự động tạo ra phản hồi. Mọi sự khác biệt phức tạp về mô hình, định dạng giao tiếp API, tốc độ phản hồi hay cơ chế truyền dữ liệu liên tục đều được che giấu kỹ lưỡng. Nhờ vậy, luồng xử lý chính luôn được giữ ở trạng thái ổn định ngay cả khi tiến hành chuyển đổi nhà cung cấp hay nâng cấp mô hình.



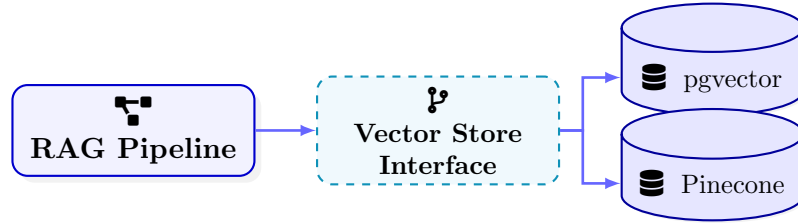
Hình 7: Trừu tượng hóa Mô hình ngôn ngữ lớn

2. Dịch vụ bản đồ và dữ liệu địa điểm: Nhóm nhận thức rõ sự khác biệt tiềm tàng về cấu trúc dữ liệu, phương thức xác thực và độ tin cậy từ các nhà cung cấp khác nhau. Giải pháp được đưa ra là trừu tượng hóa mạng lưới này thành các *nguồn dữ liệu địa điểm tiêu chuẩn*. Nhờ đó, các phân hệ cốt lõi chỉ cần làm việc với một tập dữ liệu đã được tinh gọn và chuẩn hóa thống nhất, loại bỏ sự cần thiết phải thấu hiểu logic nội bộ phức tạp của từng API riêng biệt.



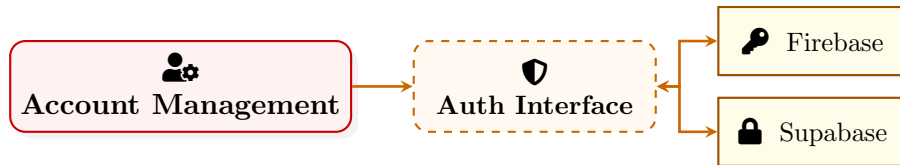
Hình 8: Trừu tượng hóa Dữ liệu Bản đồ

3. Sinh văn bản tích hợp truy xuất: Kiến trúc tổng thể không phụ thuộc chặt chẽ vào cách thức vận hành bên trong của một công nghệ vector cụ thể. Thay vào đó, toàn bộ quy trình RAG được thiết kế thành một thành phần độc lập mang trọng trách truy xuất ngữ cảnh liên quan. Những chi tiết kỹ thuật chuyên sâu về mô hình vector hóa, phương thức lưu trữ vector hay cơ chế tính toán độ tương đồng đều được ẩn giấu. Điều tối quan trọng là việc nhận lại đúng ngữ cảnh cần thiết để tổng hợp thành tư vấn đáng tin cậy.



Hình 9: Trừu tượng hóa Vector Store

4. Hạ tầng quản lý danh tính và lưu trữ: Các thành phần Authentication và Storage cũng tuân thủ nguyên tắc trừu tượng hóa. Dù đây là những dịch vụ dễ biến động tùy theo môi trường triển khai, chúng vẫn được chuyển đổi thành các khái niệm cung cấp tài nguyên thuần túy. Kết quả là, mọi mô-đun nghiệp vụ đều vận hành trơn tru mà không cần trực tiếp thao tác với cấu trúc danh tính, SDK hay giao thức độc quyền của từng nền tảng.



Hình 10: Trừu tượng hóa Quản lý Danh tính

6.3. Architectural Benefits and Balance Level

Gia tăng sự linh hoạt: Việc vận dụng triệt để Abstraction mang lại sự suy giảm đáng kể mức độ phụ thuộc chéo giữa các phân hệ, tạo không gian linh hoạt tối đa để thay thế hoặc luân chuyển nhà cung cấp. Sự phân tách này giúp hệ thống trở nên dễ kiểm thử, dễ bảo trì, và mở ra tiềm năng mở rộng không giới hạn trong tương lai. Hệ quả tất yếu là rủi ro kỹ thuật giảm thiểu tối đa, đặc biệt là khi các API bên ngoài thay đổi.

Cân bằng mức độ trừu tượng: Dù mang lại giá trị to lớn, nhóm phát triển ý thức được rằng trừu tượng hóa không phải là công cụ lạm dụng bừa bãi. Việc trừu tượng hóa quá mức sẽ biến hệ thống thành một cấu trúc mơ hồ, trong khi quá ít lại khiến logic nghiệp vụ bị gấn chặt với các nền tảng. Nhóm đã thận trọng lựa chọn điểm cân bằng: *mọi lớp trừu tượng được tạo ra đều phải gắn liền với một định hướng kiến trúc rõ ràng*. Mỗi lớp trung gian chỉ tồn tại nếu nó thực sự giải quyết bài toán thay thế nhà cung cấp, chuẩn hóa dữ liệu, hoặc gia tăng khả năng mở rộng.

7. System / Algorithm Design

7.1. Architectural Design Goals

Hệ thống được thiết kế nhằm mang lại trải nghiệm khám phá các địa điểm lưu trú và ẩm thực trực quan thông qua bản đồ. Ngoài việc cung cấp thông tin, nền tảng tập trung mạnh vào sự tương tác của những người dùng đã xác thực thông qua các tính năng đánh giá, hỏi đáp, đánh dấu yêu thích và đặc biệt là cơ chế xác thực vị trí vật lý. Nền tảng kết nối hài hòa giữa dữ liệu địa điểm từ các nguồn bên ngoài và dữ liệu cộng đồng được lưu trữ bền vững bên trong. Trợ lý trí tuệ nhân tạo được tích hợp sâu vào hệ thống nhằm cung cấp các tư vấn chuẩn xác mà không làm gián đoạn hoặc cản trở thao tác của người dùng. Xét trên phương diện vận hành, hệ thống phải đảm bảo khả năng triển khai và bảo trì dễ dàng, phù hợp với năng lực quản lý của một nhóm phát triển sinh viên.

Để đạt được những mục tiêu trên, nhóm quyết định áp dụng kiến trúc Modular Monolith thay vì kiến trúc Microservices. Quyết định này giúp tối giản hóa đáng kể quy trình triển khai và môi trường local development, đồng thời giữ được một ranh giới database transaction chung duy nhất. Nhờ vậy, độ phức tạp trong vận hành được giảm tải tối đa, nhưng hệ thống vẫn đảm bảo được sự tách biệt ranh mạch giữa các lĩnh vực nghiệp vụ thông qua cấu trúc logic module ở backend.

7.2. Overall Architecture

Hệ thống được phân tầng rõ rệt thành các lớp chuyên biệt, cấu thành một khối kiến trúc Modular Monolith hoàn chỉnh.

Frontend: Sử dụng bộ công cụ React kết hợp Vite và Tailwind CSS. Lớp này đảm nhận toàn bộ giao diện tương tác, bao gồm bản đồ tìm kiếm, thẻ thông tin địa điểm, trang chi tiết địa điểm với khu vực hỏi đáp và đánh giá. Nó cũng quản lý hiển thị trạng thái có mặt vật lý, chức năng lưu địa điểm và trang hồ sơ cá nhân.

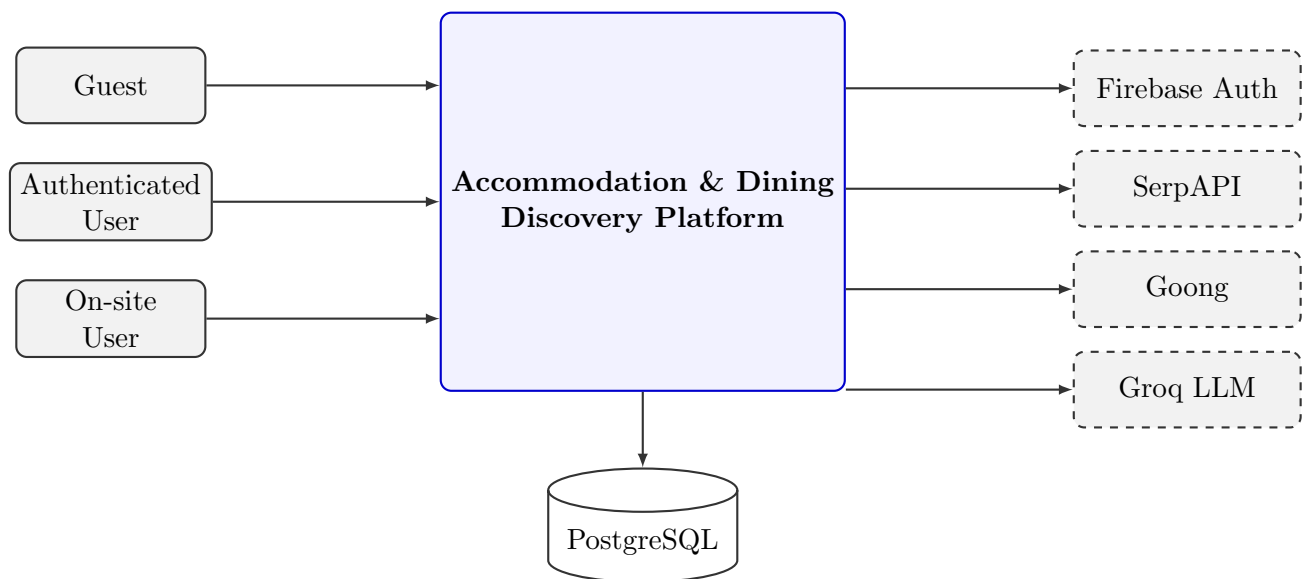
Backend/Application Layer: Được xây dựng trên nền tảng NestJS. Lớp này đóng vai trò trung tâm điều phối mọi luồng nghiệp vụ. Nó xác thực danh tính người dùng, điều hướng các luồng tìm kiếm, đồng bộ hóa dữ liệu địa điểm bên ngoài vào cơ sở dữ liệu nội bộ. Ngoài ra, lớp này quản lý toàn bộ vòng đời của các bài đánh giá, chuỗi hỏi đáp, điều phối các phản hồi từ trí tuệ nhân tạo, xác minh vị trí vật lý, và lưu trữ danh sách yêu thích.

Data Layer: Sử dụng hệ quản trị PostgreSQL kết hợp Prisma ORM. Đây là nơi lưu trữ bền vững mọi thông tin về người dùng, địa điểm, đánh giá, câu hỏi, câu trả lời, bản ghi trạng thái có mặt, danh sách yêu thích và cả lịch sử hội thoại của trợ lý thông minh.

External Integrations: Hệ thống tận dụng các dịch vụ chuyên biệt từ bên ngoài nhằm tối ưu hiệu năng: Firebase phục vụ xác thực danh tính; SerpAPI cung cấp dữ liệu tìm kiếm địa điểm; Goong đảm nhận định vị tọa độ; và mô hình LLM Groq xử lý suy luận cho trợ lý thông minh. Các lớp này tương tác chặt chẽ thông qua các giao thức mạng bảo mật, trong đó lớp Ứng dụng đóng vai trò làm lá chắn trung gian.

7.3. C4 Level 1: System Context

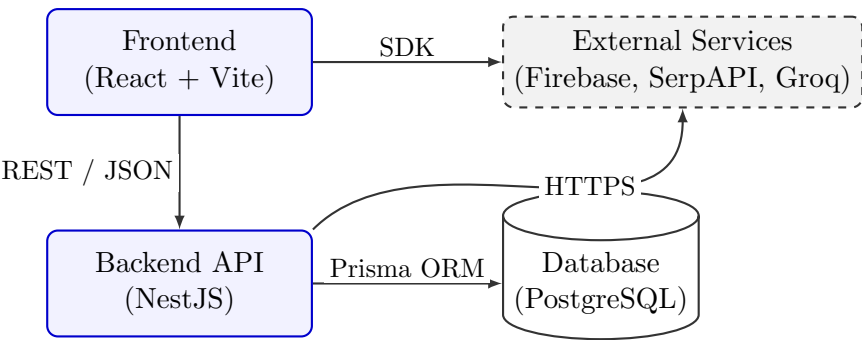
Hệ thống phục vụ ba nhóm đối tượng chính: Khách vắng lai, Người dùng đã xác thực, và Người dùng tại chỗ. Khách vắng lai có quyền xem các thông tin địa điểm và nội dung hỏi đáp công khai. Khi đã xác thực, người dùng có thể đóng góp đánh giá, đặt câu hỏi, thảo luận, lưu địa điểm và báo cáo vị trí. Người dùng tại chỗ là những cá nhân đã được hệ thống xác minh tọa độ vật lý trùng khớp với địa điểm, giúp nâng cao độ tin cậy của các câu trả lời do họ đóng góp. Hệ thống tương tác với Firebase để xác thực, SerpAPI để tìm kiếm, Goong để xử lý tọa độ, và Groq để sinh văn bản thông minh, trong khi toàn bộ dữ liệu nội tại được lưu giữ tại PostgreSQL.



Hình 11: Biểu đồ Ngữ cảnh Hệ thống (C4 Cấp 1)

7.4. C4 Level 2: Container Architecture

Hệ thống bao gồm ba container cốt lõi và các dịch vụ ngoại vi. Frontend được xây dựng bằng React và Vite, chạy trực tiếp trên trình duyệt của người dùng. Nhiệm vụ của nó là giao tiếp qua API REST định dạng JSON và đính kèm Firebase Auth Token vào các yêu cầu cần xác thực. Giao diện này kết nối trực tiếp đến Backend API, một ứng dụng NestJS tập trung toàn bộ logic nghiệp vụ, xác thực thẻ bảo mật, kết nối ngoại vi và thao tác dữ liệu. Dữ liệu được Backend API đọc ghi thông qua Database Container PostgreSQL, nơi lưu giữ chuẩn hóa toàn bộ nội dung do người dùng và trí tuệ nhân tạo tạo ra, đảm bảo tính bền vững vượt ra khỏi các kết quả tìm kiếm nhất thời.



Hình 12: Biểu đồ Kiến trúc Container (C4 Cấp 2)

7.5. Backend Modular Architecture

Máy chủ được thiết kế bám sát các business domains thay vì chia thư mục theo technical layers thuần túy. Cách tiếp cận nguyên khối phân hệ loại bỏ hoàn toàn sự phức tạp của việc trao đổi mạng nội bộ giữa các dịch vụ phân tán, nhưng vẫn duy trì tính tổ chức cực kỳ cao cấp cho mã nguồn.

Business Domain	Modules	Trách nhiệm Chính	Dữ liệu Trọng tâm
Khám phá	search, places	Điều phối truy vấn và đồng bộ hóa thông tin địa điểm.	Dữ liệu địa điểm
Định danh	users, Lớp xác thực Firebase	Quản lý hồ sơ cá nhân và kiểm chứng bảo mật.	Hồ sơ người dùng
Cộng đồng	questions, reviews, presence	Quản lý nội dung thảo luận, đánh giá và trạng thái vị trí.	Câu hỏi, Đánh giá, Trạng thái có mặt
Cá nhân hóa	saved-places, recommendations	Ghi nhớ sở thích và đề xuất địa điểm tương thích.	Danh sách yêu thích
Knowledge & AI	ai, rag	Phân loại ý định, sinh câu trả lời và tra cứu véc tơ.	Hội thoại, Vector Embeddings
Infrastructure	config, health, Truy xuất DB	Quản lý cấu hình môi trường, theo dõi sức khỏe hệ thống.	Environment Variables, Database Connection

Bảng 2: Tổ chức Business Domain tại Backend

7.6. Frontend Architecture

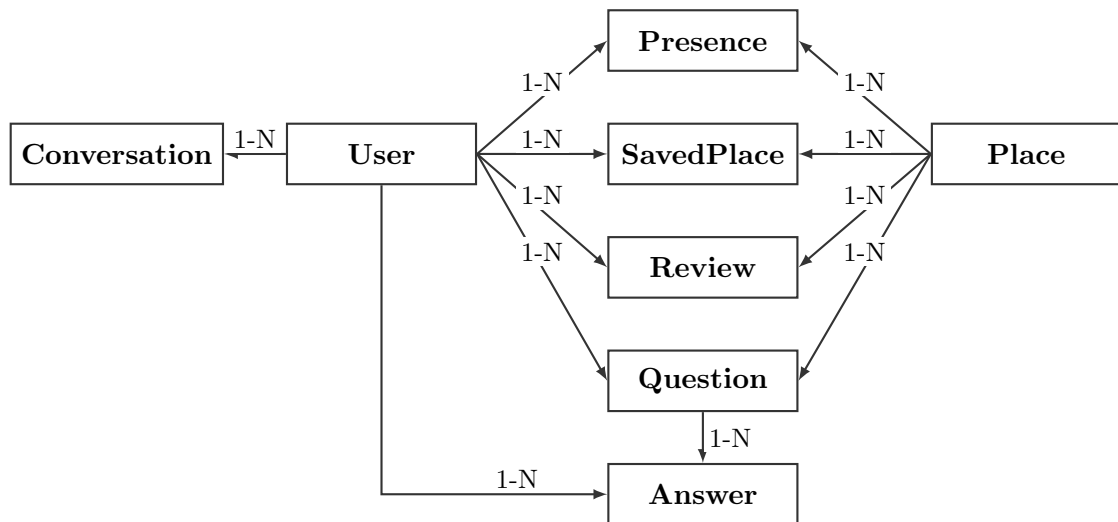
Frontend đóng vai trò là tầng điều phối quy trình làm việc và tương tác. Thay vì nhồi nhét logic nghiệp vụ, giao diện chỉ gọi các dịch vụ máy chủ và phản ánh trạng thái dữ liệu. React Router quản lý điều hướng và bảo vệ các protected routes. Context API đảm trách duy trì trạng thái toàn cục như thông tin xác thực, dữ liệu bản đồ và ngữ cảnh hội thoại.

Trang Chủ đảm nhận việc khám phá qua bản đồ, áp dụng bộ lọc và render các thẻ địa điểm cùng điểm đánh dấu. Trang Chi tiết Địa điểm đi sâu vào thông tin không gian, hiển

thị đánh giá, chuỗi hỏi đáp có gắn câu trả lời AI, nút xác nhận vị trí, nút lưu yêu thích và tích hợp luôn cửa sổ chat tư vấn chuyên biệt. Trang Đăng nhập là cửa ngõ duy nhất kết nối với hệ thống xác thực Firebase. Cuối cùng, Trang Hồ sơ cá nhân tóm tắt thông tin người dùng, trạng thái có mặt hiện tại và toàn bộ danh sách địa điểm đã lưu. Các thành phần giao diện như Bản đồ, Thẻ địa điểm, Khu vực Hỏi đáp được thiết kế độc lập, kết nối với nhau thông qua mạng lưới điều hướng và Error Boundary chặt chẽ.

7.7. Data Architecture

Cơ sở dữ liệu được chuẩn hóa nghiêm ngặt. Người dùng là trung tâm sở hữu vô số đánh giá, câu hỏi, câu trả lời, bản ghi vị trí, danh sách yêu thích và lịch sử hội thoại. Tương tự, Địa điểm là điểm neo cho các đánh giá, câu hỏi, vị trí và các tham chiếu hội thoại. Một câu hỏi chứa nhiều câu trả lời; các câu trả lời từ người dùng được liên kết nhân thân rõ ràng, trong khi các phản hồi được AI ghim lại tồn tại như những câu trả lời vô danh mang cờ hiệu hệ thống. Trạng thái vị trí lưu trữ chính xác thời điểm đến và đi. Bảng SavedPlace đóng vai trò cầu nối đa chiều giữa người dùng và địa điểm. Cấu trúc này cho phép các địa điểm ngoại vi từ API nhanh chóng hóa thân thành các thực thể bền vững trong hệ thống nội tại.



Hình 13: Sơ đồ Quan hệ Thực thể Đơn giản

7.8. Key Runtime Workflows

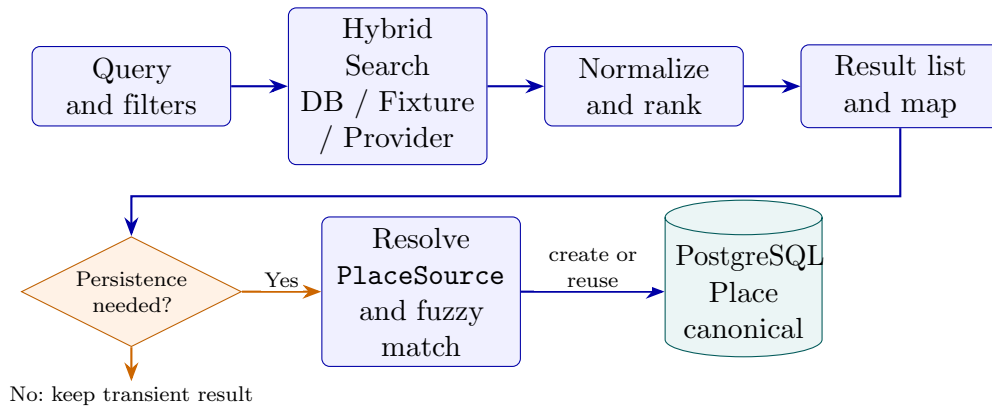
Trong số các luồng vận hành của hệ thống, ba workflow dưới đây thể hiện rõ nhất đóng góp về kiến trúc, xử lý dữ liệu và tích hợp trí tuệ nhân tạo. Các luồng hoạt động khác, dù có đóng góp vào kiến trúc hệ thống, nhưng nhóm sẽ không trình bày do độ phức tạp của các luồng đó không cao.

Workflow 1: Hybrid Search và Lazy Place Ingestion

Workflow tìm kiếm bắt đầu từ truy vấn tự nhiên hoặc bộ lọc do người dùng cung cấp. *Search Module* lựa chọn nguồn dữ liệu theo cấu hình runtime, có thể bao gồm cơ sở dữ

liệu nội bộ, bộ fixture cục bộ hoặc nhà cung cấp bên ngoài. Các kết quả từ nhiều nguồn được chuẩn hóa về một cấu trúc địa điểm thống nhất, sau đó được xếp hạng và trả về giao diện để đồng bộ danh sách với bản đồ.

Kết quả bên ngoài chưa được ghi ngay vào PostgreSQL. Chỉ khi người dùng thực hiện một tương tác cần dữ liệu bền vững, chẳng hạn lưu, đánh giá, đặt câu hỏi hoặc check-in, hệ thống mới kích hoạt *lazy ingestion*. Backend kiểm tra ánh xạ PlaceSource, tìm kiếm tương đồng theo tên, địa chỉ và tọa độ để tái sử dụng thực thể đã có; nếu không tìm thấy, hệ thống tạo một Place canonical cùng UUID nội bộ. Cách tiếp cận này tránh nhập dư thừa toàn bộ dữ liệu từ nhà cung cấp nhưng vẫn bảo đảm mọi dữ liệu cộng đồng được liên kết nhất quán.

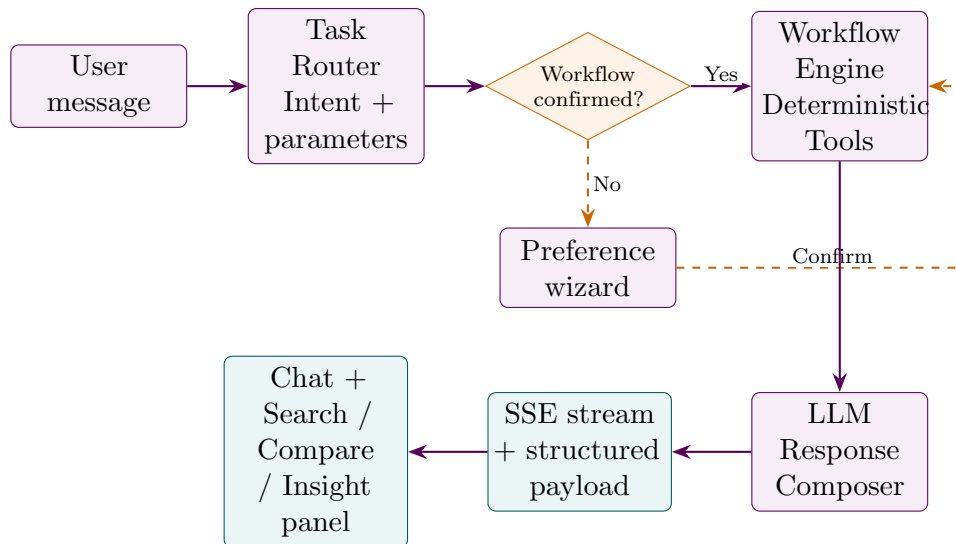


Hình 14: Hybrid Search workflow with on-demand place ingestion.

Workflow 2: AI Assistant theo Workflow có Xác nhận

AI Assistant không xử lý mọi yêu cầu như một hội thoại tự do. Khi nhận tin nhắn, *Task Router* phân loại ý định thành tìm kiếm, so sánh, phân tích một địa điểm hoặc hội thoại thông thường, đồng thời trích xuất các tham số liên quan. Với các tác vụ có ảnh hưởng đến kết quả nghiệp vụ, frontend hiển thị wizard để người dùng bổ sung tiêu chí và xác nhận trước khi thực thi.

Sau xác nhận, *Workflow Engine* chạy chuỗi công cụ xác định được khai báo trong registry. Workflow tìm kiếm lần lượt thực hiện hybrid search, geocode điểm neo và xếp hạng; workflow phân tích tổng hợp metadata, điểm xuất phát, ước lượng di chuyển và các địa điểm lân cận; workflow so sánh sử dụng tập địa điểm đã được người dùng tag. Kết quả công cụ được *Response Composer* chuyển thành câu trả lời tự nhiên hoặc payload có cấu trúc. Backend truyền phản hồi qua SSE, cho phép frontend vừa hiển thị nội dung tăng dần vừa mở đúng panel tìm kiếm, so sánh hoặc insight.

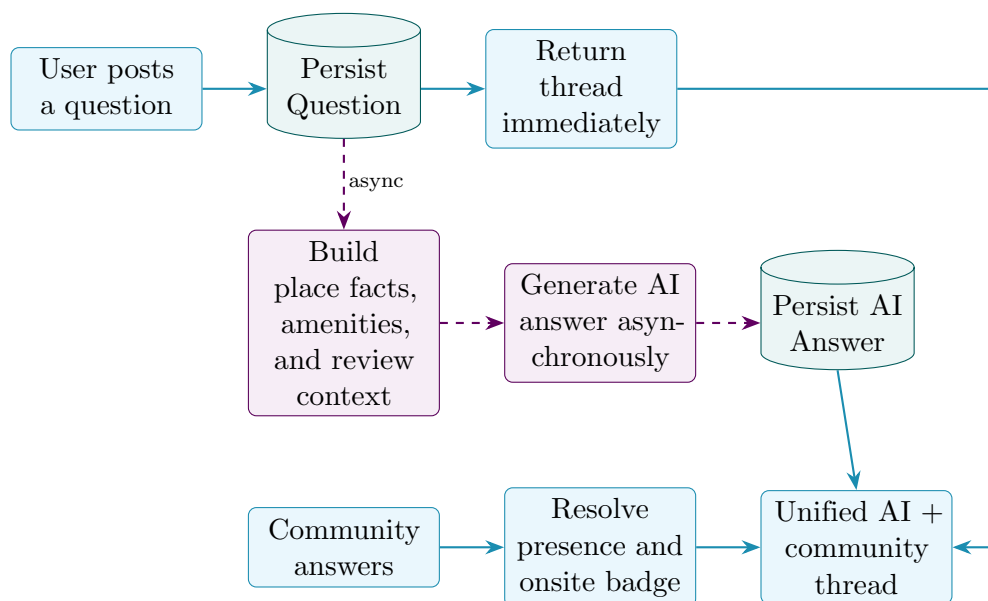


Hình 15: AI orchestration workflow using the Router–Confirmation–Engine–Composer model.

Workflow 3: Q&A Địa điểm kết hợp AI và Cộng đồng

Khi người dùng đăng câu hỏi cho một địa điểm, backend xác thực người dùng, phân giải mã địa điểm về UUID nội bộ và lưu **Question**. Thread vừa tạo được trả về ngay để giao diện cập nhật mà không phải chờ mô hình ngôn ngữ. Việc sinh câu trả lời AI diễn ra bất đồng bộ ở nền: hệ thống tập hợp metadata địa điểm, tiện ích, rating và các trích đoạn review, sau đó yêu cầu LLM tạo câu trả lời dựa trên bằng chứng hiện có.

Câu trả lời AI được lưu dưới dạng **Answer** không gắn **userId**, nhờ đó có thể tách riêng khi dựng thread. Song song với AI, người dùng cộng đồng tiếp tục đăng câu trả lời thông thường; khi đọc thread, backend đối chiếu **Presence** để đánh dấu tác giả đang onsite. Workflow này kết hợp tốc độ phản hồi của AI với tri thức thực địa của cộng đồng mà không làm thao tác đăng câu hỏi bị chặn.



Hình 16: Asynchronous Q&A workflow combining AI answers with onsite community knowledge.

7.9. Security and Access Control

Bảo mật hệ thống ủy thác cho Firebase ở client-side. Bất kỳ tác vụ cập nhật dữ liệu nào cũng phải đính kèm thẻ xác thực Firebase để NestJS xác thực qua Admin SDK. Các thao tác đọc thông tin công cộng được nói lỏng nhằm tối ưu trải nghiệm, nhưng mọi thao tác ghi, đặc biệt là quyền xóa đánh giá hay xóa câu hỏi, đều bị kiểm tra quyền sở hữu cực kỳ gắt gao. Khi một câu hỏi bị xóa, hệ thống kích hoạt cơ chế cascade delete dọn dẹp sạch sẽ các câu trả lời liên quan, đảm bảo không lưu lại rác dữ liệu. Trạng thái vị trí cũng có thể tự do xóa bỏ để bảo vệ quyền ẩn danh.

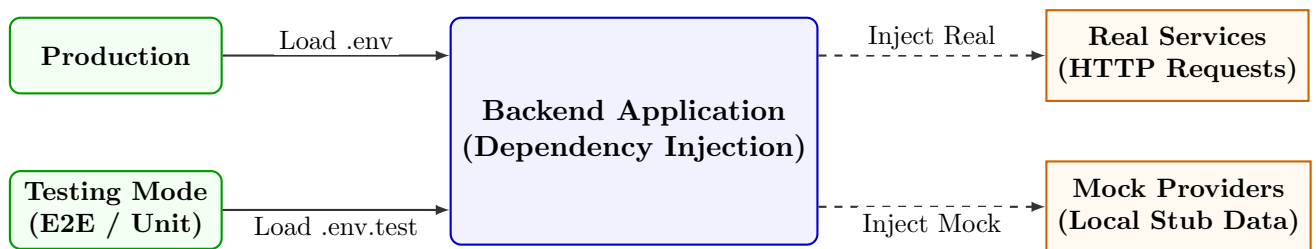
7.10. Runtime Configuration and Environment Strategy

Hệ thống áp dụng chiến lược Runtime Configuration thông qua Dependency Injection của NestJS để chuyển đổi mượt mà giữa môi trường Testing và Production. Việc branching logic được hạn chế tối đa; thay vào đó, toàn bộ cấu hình được trừu tượng hóa và nạp động từ các tệp `.env` tương ứng.

Đặc biệt ở chế độ Testing Mode, cơ chế phát triển được tối ưu hóa như sau:

- **Mocking External APIs:** Thay vì gửi request thực tế tốn chi phí và phụ thuộc mạng lưới, các dịch vụ ngoại vi (SerpAPI, Goong, Groq LLM) được hệ thống tự động inject các Mock Providers trả về kết quả định trước.
- **Isolated Test Database:** Chế độ kiểm thử sử dụng một schema cơ sở dữ liệu hoàn toàn tách biệt. Dữ liệu được teardown (xóa sạch) sau mỗi chu kỳ chạy E2E Test, đảm bảo tính tất định tuyệt đối cho các kịch bản kiểm thử.
- **Ngắt lưu trữ rác:** Các chức năng ghi log hội thoại AI, xác thực Firebase thực tế hay đồng bộ dữ liệu tĩnh được vô hiệu hóa hoặc thay thế bằng cơ chế nội bộ để tăng tốc độ chạy test.

Cơ chế này cho phép các kỹ sư liên tục chạy unit test và e2e test ở môi trường cục bộ một cách độc lập, ổn định và không lo ngại giới hạn rate-limit của API bên ngoài.



Hình 17: Cơ chế Dependency Injection theo Môi trường Kiểm thử

7.11. Architectural Decisions and Trade-offs

Quyết định	Động lực	Lợi ích	Đánh đổi
Modular Monolith	Cần kiểm soát ranh giới rõ ràng nhưng thiếu nguồn lực vận hành phân tán.	Dễ dàng local deployment, dùng chung một kết nối cơ sở dữ liệu.	Không thể linh hoạt thay đổi quy mô từng phân hệ một cách độc lập.
Giao tiếp REST JSON	Tìm kiếm tiêu chuẩn công nghiệp an toàn và phổ biến nhất.	Hệ sinh thái công cụ dồi dào, gỡ lỗi mạng cực kỳ đơn giản.	Lãng phí một phần băng thông so với cấu trúc nhị phân nguyên thủy.
Xác thực Firebase	Tránh rủi ro tự quản lý mật khẩu mã hóa.	Chuẩn bảo mật cực cao, rút ngắn thời gian phát triển.	Bị phụ thuộc hoàn toàn vào dịch vụ đám mây của Google.
Đồng bộ Địa điểm Động	Cần lưu đánh giá cho địa điểm từ API ngoài.	Biến dữ liệu tạm thời thành dữ liệu bền vững nội bộ.	Bơm một lượng dữ liệu thụ động vào cơ sở dữ liệu.
Sinh câu trả lời AI bất đồng bộ	Tránh tình trạng chờ đợi quá lâu khi nạp ngữ cảnh vào LLM.	Giao diện phản hồi ngay lập tức cho người hỏi.	Phải bổ sung logic làm mới giao diện sau khi tạo xong.

Bảng 3: Các Quyết định Kiến trúc và Sự Đánh đổi

7.12. Current Limitations and Future Improvements

Kiến trúc bộc lộ một số giới hạn nhất định trong phạm vi đề án hiện tại. Tính năng tính chỉnh bộ lọc tìm kiếm nâng cao theo phân loại địa điểm vẫn đang trong giai đoạn hoàn thiện. Luồng điều hướng và giao diện chỉ đường chưa đạt được sự mượt mà tối đa. Hệ thống cũng chịu sự ràng buộc bởi các dịch vụ cung cấp dữ liệu tìm kiếm, định vị và xử lý ngôn ngữ bên ngoài. Hơn nữa, chất lượng tư vấn của AI phụ thuộc tuyến tính vào lượng dữ liệu thực tế thu thập được; do đó, cơ chế kiểm duyệt và xếp hạng bình chọn cho các câu hỏi sẽ cần được bổ sung để nâng cấp chất lượng tri thức cộng đồng.

7.13. Summary

Kiến trúc hệ thống thể hiện sự rạch ròi tuyệt đối giữa việc render giao diện, xử lý luồng nghiệp vụ cốt lõi, lưu trữ dữ liệu bền vững và giao tiếp ngoại vi. Việc ứng dụng linh hoạt mô hình Modular Monolith giúp nhóm phát triển đạt được sự cân bằng hoàn hảo giữa tính dễ bảo trì và sự tinh giản trong vận hành. Nhờ những cơ chế đột phá như xác thực tọa độ vị trí vật lý và quy trình đồng bộ dữ liệu ngoại vi theo thời gian thực, nền tảng không chỉ là một công cụ tìm kiếm đơn thuần mà đã thực sự chuyển mình thành một mạng lưới tri thức cộng đồng an toàn, đáng tin cậy và mang đậm tính cá nhân hóa sâu sắc.

8. Implementation

8.1. Implementation Strategy

Thay vì lập trình tự phát theo từng tính năng lẻ tẻ, nhóm tiếp cận việc hiện thực hóa bằng cách bám sát luồng nghiệp vụ cốt lõi đã thiết kế. Mục tiêu đầu tiên là xây dựng nhanh một bản MVP (Minimum Viable Product) để kiểm chứng ý tưởng. Các thành phần được chia nhỏ thành những module độc lập, cho phép mỗi thành viên phát triển và kiểm thử cục bộ trước khi tích hợp vào hệ thống chung. Chiến lược này giúp nhóm kiểm soát rủi ro, đẩy nhanh tiến độ và khoanh vùng lỗi hiệu quả.

8.2. From Design to Implementation

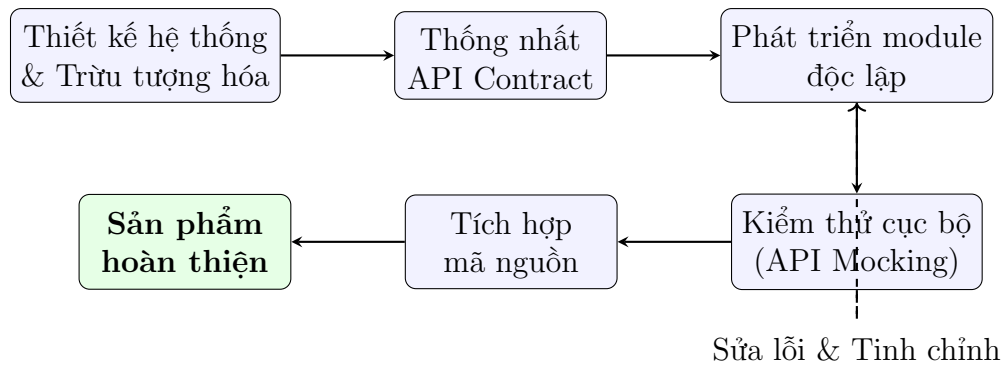
Bản chất của quá trình triển khai là việc ánh xạ trực tiếp các quyết định từ giai đoạn thiết kế, phân rã và trừu tượng hóa thành mã nguồn thực tế (Bảng 4).

Quyết định thiết kế	Chiến lược hiện thực hóa	Lý do / Tác động
Lưu trữ và tìm kiếm vector	Sử dụng PostgreSQL kết hợp extension pgvector thông qua các truy vấn Raw SQL.	Tính toán độ tương đồng ngay trong cơ sở dữ liệu quan hệ, tránh duy trì thêm một hệ thống độc lập tốn kém.
Kiến trúc hướng Module	Tổ chức mã nguồn Backend thành các module NestJS biệt lập.	Tăng tính đóng gói, dễ dàng kiểm thử và chia việc giữa các thành viên mà không gây xung đột mã.
Trừu tượng hóa nhà cung cấp	Tạo các giao diện chuẩn cho Map Provider và hệ thống LLM.	Cho phép hệ thống dễ dàng thay đổi nhà cung cấp (ví dụ đổi từ OSM sang Goong) mà không sửa logic lõi.
Tách biệt luồng AI	Triển khai luồng xử lý 4 lớp (Router → Orchestrator → Tools → Composer).	Quản lý chặt chẽ hành vi của AI, đảm bảo tính tất định và hạn chế hiện tượng Hallucination.

Bảng 4: Ánh xạ từ thiết kế sang triển khai

8.3. Team Implementation Workflow

Để phối hợp nhịp nhàng, nhóm thiết lập một quy trình chuẩn hóa: thống nhất quy ước giao tiếp dữ liệu (API Contract) giữa các module, lập trình độc lập, kiểm thử cục bộ, và cuối cùng là tích hợp mã nguồn (Merge). Sơ đồ 18 minh họa vòng lặp liên tục này, đảm bảo mỗi đoạn mã được đưa lên nhánh chính đều đã trải qua khâu kiểm chứng, giữ cho dự án luôn trong trạng thái sẵn sàng vận hành.



Hình 18: Quy trình triển khai mã nguồn liên tục của nhóm

8.4. Core Implemented Parts

Hệ thống được lắp ghép từ bốn khối thành phần chính, mỗi khối đảm nhận một mắt xích trong luồng xử lý:

- **Giao diện người dùng (Frontend):** Cung cấp trải nghiệm tương tác bản đồ trực quan và khung chat thời gian thực, là nơi tiếp nhận ý định của người dùng.
- **Xử lý nghiệp vụ (Backend Core):** Điều phối dữ liệu trung tâm, áp dụng mẫu thiết kế Dependency Injection để tăng tính tái sử dụng và dễ dàng bảo trì.
- **Lưu trữ dữ liệu (Database):** Đóng vai trò xương sống cho tính năng truy xuất thông tin, lưu trữ cả dữ liệu quan hệ truyền thống lẫn các Vector Embeddings.
- **Động cơ AI (AI Engine):** Trái tim của hệ thống, nơi các câu lệnh tự nhiên được phân tích ý định và chuyển hóa thành các thao tác tra cứu cụ thể trên bản đồ.

8.5. Technical Challenges and Solutions

Xây dựng một hệ thống kết hợp bản đồ số và tác tử AI đem lại nhiều bài toán hóc búa. Bảng 5 đúc kết những thách thức cốt lõi và cách nhóm tháo gỡ vấn đề.

Thách thức	Nguyên nhân	Giải pháp	Tác động
Dữ liệu từ AI thiếu nhất quán	Do tính phi tất định của các mô hình ngôn ngữ lớn (LLM).	Áp dụng định nghĩa cấu trúc nghiêm ngặt kết hợp phân tách luồng suy luận và thực thi.	Nhận diện ý định chuẩn xác, hệ thống không bị treo do lỗi phân tích cú pháp (Parsing Error).
Truyền tải dữ liệu thời gian thực	Giao thức Server-Sent Events (SSE) tiêu chuẩn chỉ hỗ trợ phương thức GET.	Sử dụng luồng xử lý SSE tùy chỉnh kết hợp phương thức POST để bảo mật thông tin.	Phản hồi mượt mà từng chữ nhưng vẫn đảm bảo tính an toàn dữ liệu.
Dịch vụ bản đồ thiếu ổn định	API của các nhà cung cấp thỉnh thoảng bị nghẽn mạng hoặc quá tải.	Áp dụng cơ chế Fallback kết hợp xử lý bất đồng bộ.	Ứng dụng duy trì tính bền bỉ, tìm kiếm không bị gián đoạn.
Tính toán vector trên ORM	Các ORM truyền thống không hỗ trợ tốt phép toán hình học đa chiều.	Sử dụng Raw Query để can thiệp trực tiếp vào tính toán độ tương đồng.	Giữ được sức mạnh toán học của cơ sở dữ liệu chuyên dụng.

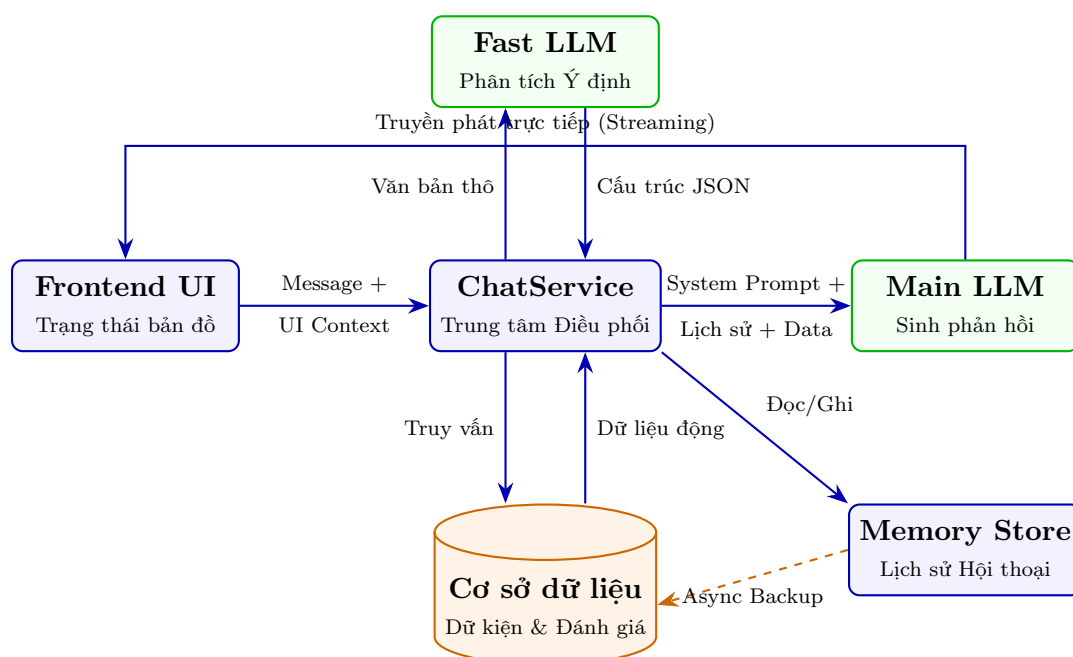
Bảng 5: Các rào cản kỹ thuật và cách giải quyết

8.6. Quản lý ngữ cảnh giao tiếp cho mô hình ngôn ngữ

Để mô hình ngôn ngữ lớn (LLM) có thể thực hiện trọn vẹn vai trò của một trợ lý du lịch ảo, rào cản lớn nhất không nằm ở sức mạnh suy luận của mô hình, mà là cách thiết kế luồng cung cấp ngữ cảnh đúng, đủ và liên tục. Nhóm đã xây dựng một cơ chế quản lý ngữ cảnh đa tầng xuyên suốt từ giao diện người dùng đến máy chủ xử lý:

- **Đóng gói trạng thái giao diện trực tiếp:** Khi người dùng nhấn tin, dữ liệu gửi đi không chỉ là văn bản thô. Tầng giao diện tự động thu thập các địa điểm đang được chọn, luồng công việc hiện tại, và các giới hạn ngân sách. Các thông tin này được đính kèm vào khối dữ liệu tải lên. Điều này giúp LLM có khả năng "nhìn" thấy bản đồ, hiểu được người dùng đang quan tâm đến địa điểm nào mà không cần họ phải gọi tên cụ thể.
- **Trích xuất ý định bằng luồng AI song song:** Thay vì sử dụng các biểu thức chính quy để bóc tách từ khóa tìm kiếm, hệ thống định hướng đầu vào bằng một lời gọi LLM phụ trợ siêu tốc. Lời gọi này có nhiệm vụ duy nhất là phân tích văn bản và trả về một cấu trúc dữ liệu JSON bao gồm: cờ xác nhận tìm kiếm, vị trí, phân loại (chỗ ở, ăn uống, tham quan) và ngân sách. Nhờ vậy, hệ thống có thể nhận diện yêu cầu di chuyển bản đồ từ những câu hội thoại tự nhiên nhất.

- **Tiêm dữ liệu động chống hiện tượng Hallucination:** Để trả lời chuyên sâu về một địa điểm cụ thể, hệ thống tuyệt đối không để LLM tự suy diễn dựa trên kiến thức cũ. Dịch vụ xử lý trò chuyện sẽ tự động trích xuất từ cơ sở dữ liệu một khối thông tin chứa các dữ kiện xác thực: nguồn cung cấp, danh mục, điểm đánh giá trung bình, các tiện ích, và đặc biệt là tối đa 5 trích đoạn đánh giá thực tế nổi bật nhất. Chỉ thị gốc của hệ thống (System Prompt) được thiết lập nghiêm ngặt: yêu cầu LLM chỉ sử dụng các bằng chứng vừa được cung cấp, trả lời bằng đại từ xưng hô gần gũi ("mình "bạn"), và thẳng thắn từ chối nếu thông tin không đủ để đưa ra kết luận.
- **Quản lý bộ nhớ hội thoại dạng cửa sổ trượt:** Nhằm tối ưu hóa giới hạn ngữ cảnh đầu vào và giảm thiểu chi phí API, lịch sử trò chuyện được kiểm soát chặt chẽ. Hệ thống duy trì một bộ nhớ đệm cục bộ với thời gian sống là 3600 giây, và tự động cắt xén chỉ giữ lại đúng 20 tin nhắn gần nhất. Đồng thời, cấu hình hệ thống cho phép đồng bộ hóa dữ liệu này vào cơ sở dữ liệu PostgreSQL một cách phi đồng bộ, đảm bảo tính liên tục của các phiên hội thoại khi tải lại trang mà không làm chậm luồng phản hồi chính.



Hình 19: LLM Context Management Workflow — Minh họa luồng dữ liệu quản lý ngữ cảnh từ giao diện đến LLM.

Việc luân chuyển ngữ cảnh một cách khéo léo và có chủ đích này chính là chìa khóa giúp một hệ thống AI phi trạng thái trở thành một trợ lý ảo thông minh, biết nhớ bối cảnh và phản hồi sắc bén theo thời gian thực.

8.7. Summary

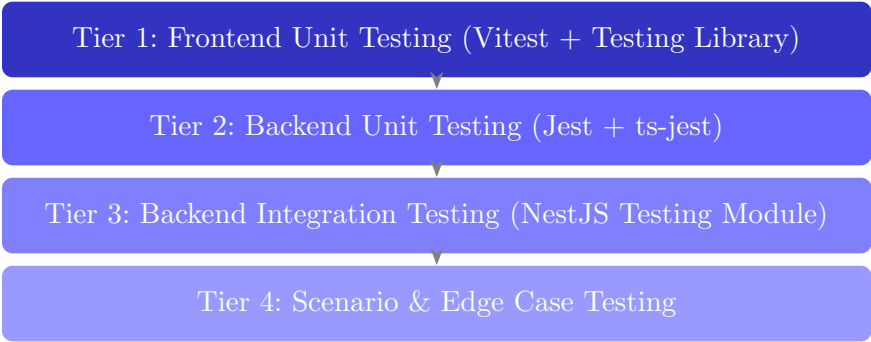
Tóm lại, phần triển khai đã phản ánh chân thực năng lực chuyển hóa các mô hình lý thuyết trừu tượng thành một hệ thống vận hành thực tiễn. Vượt qua những rào cản về

công nghệ và kiến trúc, chiến lược chia nhỏ để trị kết hợp cùng quy trình cải tiến liên tục đã giúp nhóm đúc kết thành một nền tảng vững vàng, dễ bảo trì và mở ra nhiều cơ hội nâng cấp về sau.

9. Testing

9.1. Testing Strategy and Methodology

Nhóm phát triển áp dụng chiến lược kiểm thử đa tầng nghiêm ngặt, bao phủ từ các đơn vị mã nguồn nhỏ nhất cho đến tích hợp toàn hệ thống. Mục tiêu cốt lõi không chỉ là phát hiện lỗi sớm mà còn đảm bảo kiến trúc hệ thống không bị suy thoái trong quá trình bảo trì và mở rộng.



Nguyên tắc thiết kế Test: Toàn bộ tầng dữ liệu và các dịch vụ ngoại vi (Firebase, Groq, SerpAPI) đều được giả lập hoàn toàn trong các bài kiểm thử đơn vị, nhằm đảm bảo tính độc lập và tốc độ thực thi. Ngược lại, kiểm thử tích hợp sử dụng chính cơ chế Dependency Injection của NestJS để liên kết các module thật, đảm bảo dữ liệu luân chuyển nhất quán qua toàn bộ vòng đời của một yêu cầu mạng.

9.2. Execution Results and Statistics

Quá trình đánh giá được thực hiện thông qua hệ thống tích hợp liên tục cục bộ vào ngày 19/06/2026. Kết quả thống kê chi tiết cho thấy tỷ lệ hoàn thành ở mức rất cao so với độ phức tạp của một hệ thống nguyên khối phân hệ:

Component	Suites / Files	Total Tests	Pass Rate	Time
Backend (Jest)	46 / 69 passed	337 / 370 passed	91.1%	67.1s
Frontend (Vitest)	57 / 64 passed	196 / 207 passed	94.7%	41.1s

Bảng 6: Thống kê kết quả thực thi các bài kiểm thử

Frontend (207)	196 Pass (94.7%)	11 Fail
Backend (370)	337 Pass (91.1%)	33 Fail

9.3. Quality Assurance and Deep Dive Evaluation

Qua đánh giá tổng thể kể trên, chất lượng hiện tại của nền tảng được đánh giá rất tích cực, tuy nhiên vẫn tồn tại một số nợ kỹ thuật cần được giải quyết triệt để.

9.3.1 Điểm mạnh

- **Khả năng mở rộng không tạo hồi quy:** Đợt bổ sung 16 tập tin kiểm thử mới nhất (bao gồm cả phân tích không gian, cấu hình runtime, hệ thống AI Orchestrator và Frontend Router) đã **vượt qua 100% các ca thử nghiệm** mà không làm phá vỡ bất kỳ logic lõi nào. Điều này chứng minh cấu trúc của Backend đang làm rất tốt việc cô lập phạm vi ảnh hưởng.
- **Độ bao phủ của hệ thống thông minh:** Toàn bộ hệ sinh thái truy xuất tăng cường, phân tích ngôn ngữ tự nhiên, và luồng thực thi trí tuệ nhân tạo đều hoạt động hoàn hảo dưới môi trường giả lập. Máy trạng thái phân giải ý định người dùng không bị sai lệch ngay cả khi nhận vào các chuỗi ngôn ngữ tự nhiên phức tạp.
- **Tốc độ phản hồi:** Hoàn tất hơn 570 bài kiểm tra cho cả hai đầu hệ thống trong vòng xấp xỉ hai phút là một chỉ số hiệu suất ấn tượng, tạo tiền đề lý tưởng cho việc thiết lập các luồng tự động hóa triển khai nghiêm ngặt sau này.

9.3.2 Điểm hạn chế

Toàn bộ các ca thất bại hiện tại (33 lỗi ở backend và 11 lỗi ở frontend) đều là **các lỗi tồn đọng từ các đợt tái cấu trúc trước**, hoàn toàn không phải lỗi logic nghiệp vụ cốt lõi mới phát sinh. Phân tích nguyên nhân gốc rễ chỉ ra ba vấn đề chính:

1. **Mất đồng bộ cơ chế Dependency Injection:** Hơn 9 ca lỗi ở Backend xuất phát từ việc các bộ kiểm thử Controller sử dụng hệ thống Guard bảo mật nhưng kịch bản kiểm thử lại chưa cung cấp dịch vụ cấu hình vào trong khối khởi tạo của module kiểm thử.
2. **Đối tượng giả lập không hoàn chỉnh:** Tại Frontend, các dịch vụ liên quan trực tiếp đến cơ sở dữ liệu thời gian thực thất bại do đối tượng giả lập không phản ánh đúng chữ ký hàm của SDK mới nhất. Tương tự ở Backend, một số luồng thao tác với trí tuệ nhân tạo bị gián đoạn trong môi trường kiểm thử do việc giả lập thiếu các thuộc tính bắt buộc.
3. **Mất đồng bộ chuẩn định dạng dữ liệu và mã nguồn cũ:** Sự vắng mặt của các tệp khai báo kiểu dữ liệu và sự tồn tại của các tập tin kiểm thử cũ (sử dụng

API cũ đã bị xóa bỏ) gây ra các lỗi biên dịch hoặc lỗi tham chiếu. Ngoài ra, một component hiển thị tại frontend đang gặp lỗi do bộ công cụ kiểm thử cố gắng tìm một chuỗi văn bản thuần thay vì sử dụng nhãn nhận dạng hỗ trợ tiếp cận vốn đã được áp dụng để tối ưu giao diện.

Kết luận rà soát: Hệ thống lõi hoạt động ổn định và có mức độ hoàn thiện cao, sẵn sàng cho việc đóng gói và triển khai thực tế. Việc dọn dẹp các ca thất bại không đòi hỏi thiết kế lại thuật toán, mà thuần túy là quá trình đồng bộ hóa cấu hình môi trường kiểm thử với cấu trúc mã nguồn đã được nâng cấp.

10. Logbook: Plan and Actual Workload

10.1. Detail of Plan Workload

STT	MSSV	Họ và tên	Các công việc dự kiến
1	24120033	Đào Tiến Đạt	
2	24120040	Dương Hoài Đức	
3	24120304	Trần Thị Bích Hạnh	
4	24120310	Nguyễn Đình Hiếu	
5	24120483	Hoàng Văn Trường	

Bảng 7: Chi tiết kế hoạch phân công

10.2. Actual Workload

STT	MSSV	Họ và tên	Công việc thực tế
1	24120033	Đào Tiến Đạt	
2	24120040	Dương Hoài Đức	
3	24120304	Trần Thị Bích Hạnh	
4	24120310	Nguyễn Đình Hiếu	
5	24120483	Hoàng Văn Trường	

Bảng 8: Chi tiết khối lượng công việc thực tế

10.3. Workload Distribution

STT	MSSV	Họ và tên	Tỷ lệ đóng góp	Số giờ làm việc
1	24120033	Đào Tiến Đạt		
2	24120040	Dương Hoài Đức		
3	24120304	Trần Thị Bích Hạnh		
4	24120310	Nguyễn Đình Hiếu		
5	24120483	Hoàng Văn Trường		

Bảng 9: Phân bổ khối lượng công việc của các thành viên

10.4. List of tools used

Dưới đây là danh sách các công cụ, công nghệ và dịch vụ ngoại vi đã được sử dụng trong suốt quá trình thiết kế, phát triển và kiểm thử dự án:

Mục đích sử dụng	Công cụ sử dụng
Giao tiếp và liên lạc	Zalo, Notion
Quản lý dự án	Notion
Quản lý mã nguồn	Git, GitHub
Quản lý tài liệu	Overleaf, Prism
Lập trình (AI Agents)	Antigravity CLI (agy), Codex, Cursor
Triển khai và kiểm thử	Vercel, Docker, Postman

Bảng 10: Danh sách công cụ hỗ trợ dự án

11. AI usage and declaration

12. Conclusion and Future Work

12.1. Conclusion

Nhìn lại toàn bộ hành trình thực hiện đồ án, nhóm đã nghiên cứu, thiết kế và phát triển thành công một hệ thống tìm kiếm và gợi ý địa điểm lưu trú tích hợp Trí tuệ Nhân tạo. Khởi điểm từ việc áp dụng bốn trụ cột của Tư duy Máy tính để phân rã bài toán, nhóm đã chuyển hóa thành công các yêu cầu trừu tượng thành một hệ thống thực tế hoàn chỉnh. Việc áp dụng kiến trúc Modular Monolith kết hợp cùng các công nghệ hiện đại như NestJS, React, cơ sở dữ liệu vector pgvector, và hệ thống truy xuất thông tin RAG thông qua Groq LLM đã chứng minh được tính hiệu quả và khả năng mở rộng của nền tảng.

Sản phẩm cuối cùng không chỉ đáp ứng các chức năng cốt lõi đã đề ra—như tìm kiếm bằng ngôn ngữ tự nhiên, so sánh địa điểm và trích xuất thông tin tự động—mà còn đảm bảo tính nhất quán về mặt kiến trúc phần mềm và quy trình kiểm thử.

12.2. Team Reflections and Lessons Learned

Đồ án này là một thách thức lớn nhưng đồng thời cũng là một cơ hội quý báu để các thành viên trong nhóm cọ xát với môi trường phát triển phần mềm chuyên nghiệp. Từ quá trình thực tiễn, nhóm đã đúc kết được nhiều bài học sâu sắc:

- **Tầm quan trọng của khâu thiết kế kiến trúc:** Việc thay đổi chiến lược từ Microservices sang Modular Monolith ở giai đoạn đầu đã dạy cho nhóm bài học đắt

giá về việc lựa chọn kiến trúc phù hợp với quy mô dự án và năng lực đội ngũ. Một thiết kế tốt sẽ giúp giảm thiểu nợ kỹ thuật ở các giai đoạn sau.

- **Cân bằng giữa công nghệ mới và tính ổn định:** Việc ứng dụng các công nghệ mới như pgvector hay mô hình ngôn ngữ lớn (LLM) đòi hỏi sự kiên nhẫn trong việc nghiên cứu tài liệu. Nhóm nhận ra rằng AI không phải là "phép thuật" giải quyết mọi vấn đề, mà cần được tích hợp có kiểm soát thông qua Prompt Engineering chặt chẽ để tránh hiện tượng Hallucination.
- **Kỷ luật trong làm việc nhóm và quản lý mã nguồn:** Quá trình vận hành dự án với nhiều thành viên đã chứng minh giá trị của việc tuân thủ các quy chuẩn Git, kiểm thử liên tục, và giao tiếp minh bạch. Sự hỗ trợ từ các công cụ AI Agents (như Antigravity CLI) cũng đóng vai trò xúc tác lớn, giúp nhóm tối ưu hóa hiệu suất lập trình.

Nhận xét chung về cá nhân và tập thể, mặc dù dự án vẫn còn một số điểm có thể tối ưu hóa thêm về mặt hiệu năng xử lý song song, nhưng kết quả đạt được đã vượt qua sự kỳ vọng ban đầu. Qua đồ án này, mỗi cá nhân trong nhóm đều đã tự nâng tầm tư duy hệ thống và hoàn thiện kỹ năng giải quyết bài toán cốt lõi.

12.3. Future Development Directions

Để phát triển dự án từ một nguyên mẫu thành một sản phẩm thương mại hoàn chỉnh, nhóm đề xuất các hướng cải tiến sau:

- **Cải tiến mô hình RAG và Fine-tuning LLM:** Tối ưu hóa chiến lược truy xuất Semantic Search, áp dụng các kỹ thuật phân mảnh văn bản thông minh hơn và tiến tới fine-tune một mô hình LLM chuyên biệt cho lĩnh vực du lịch nhằm tăng tốc độ phản hồi.
- **Mở rộng sang kiến trúc Microservices thực thụ:** Khi hệ thống cần phục vụ lượng người dùng lớn hơn, việc tách các module (như AI Engine, Authentication, Core Services) thành các dịch vụ độc lập sẽ được cân nhắc triển khai cùng với hệ thống điều phối container.
- **Tối ưu hóa UI/UX và phân tích dữ liệu:** Nâng cấp trải nghiệm người dùng với các giao diện cá nhân hóa sâu hơn, tích hợp hệ thống tracking hành vi để hệ thống có khả năng gợi ý địa điểm lưu trú một cách chủ động.